Generation Phase - Prompt Engineering

We tested different prompts but documented only the four listed below as they provided stable results. Other prompts were either producing inconsistent outputs or providing incomplete information, which is why they were not included.

"P1": "Answer the question based only on the following context:\n\n{context}\n\n---\n\nAnswer the question based on the above context.\n\nQuestion: {query}\n\nResponse:\n- Answer: [Provide the detailed answer here]\n- Conclusion: [Yes/No/Not Applicable]\n"

"P2": "Act as a professional assistant in the field of software development, you need to give precise and short answers to respond to the question that I gave.\nI will take the corresponding text snippet from the design file of the software development, and you need to use a certain format and \"yes/no/not applicable\" to answer the question.\n\nMy Input would be:\n\"\"\nAnswer the question based only on the following context:\n<context>\n\nQuestion:\n<question>\n\"\"\n\nFor My Input:\n<context>: Five paragraphs excerpted from my design document for software development.\n<question>: I'll ask you if this uses a certain technology to support a certain green practice. For example, the question could be: \"Does the application/framework use content delivery networks (CDNs) to minimize recomputation or fetching of static data?\"\n\nYour Answer must adhere to this format:\n\"\"\nResponse:\nJudgement: Print <Yes> / <No> / <Not Applicable> only.\nExplanation: <The description of the reason for the judgment above>\n\"\"\n\nFor Your Answer:\n\nIn judgment,\n<Yes> means that in the context of my question, there exists a technology or green practice that is relevant to the question.\n<No> means that in the context of my question, there is no technology or green practice that is relevant to the question.\n<Not Applicable> means that in the context of my question, this application is not applicable to this technique or to the green practice, e.g., applications that need to focus on real-time feedback, such as online games, are not applicable to the green practice of \"cache static data\".\n\nIn Explanation, you need to explain the judgment you made above in less than 3 sentences.\n\nNow, here's my input:\nAnswer the question based only on the following context:\n{context}\n\nQuestion:\n{query}"

"P3":"Act as a professional assistant in software development and use this as context '{context}' to answer this question '{query}'",

"P4":"Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request. \n\n### Instruction: Using the context below, answer this question: '{query}'\n\n### Input:{context}", Initially, our goal was to determine if we could achieve stable responses using prompts alone, without fine-tuning.

After identifying these four prompts, we began fine-tuning the model and obtained varying results for each prompt, as follows:

The prompt P1 produced good results initially, but after fine-tuning the model, we noticed variations in the output structure. We realized that since we were fine-tuning the model with a dataset that already followed our desired structure, explicitly stating the structure in the prompt became redundant.

The prompt P2 was too lengthy, and the responses sometimes deviated from the expected format. We observed that the structure was inconsistent, and occasionally the LLM would repeat sections of the prompt itself. This could be attributed to the relatively small context window being overshadowed by the lengthy prompt.

While the prompt P4 appears simpler, words like "instruction," "task," and "complete the request" influenced the answers in unintended ways. For example, responses would sometimes begin with phrases like "The request can't be completed because..."

We ultimately selected **P3**, which delivered promising results both before and after fine-tuning.

This process involved considerable trial and error. While there might be better prompts available, **P3** proved sufficient for our prototype by providing consistent results.

Additionally, we explored optimizing Phi-3 with DSPy. DSPy is a Python framework designed for optimizing LLM prompts and weights. It uses custom optimizers to adjust prompts and weights to maximize specific metrics, providing a high-level API that requires only the metric and a training set.. In our initial experiments, we used DSPy's built-in answer exact match metric. However, the performance of the optimized LLM was suboptimal when evaluated, which was expected since the metric only considered whether the output of the LLM exactly matched the ground truth. Due to the absence of a suitable metric for our use case—and the closest we got was attempting to use an LLM as a judge where a more powerful LLM is used to train a smaller one—we ultimately abandoned this approach due to limited resources and the inadequacy of available metrics.